OPERATOR PDF EDITION

Operator Guides

Practical runbook for The Ontology Machine: surfaces, paths, expected signs, failures, recovery, and handover.

Core rule: do not guess across ownership boundaries. Inspect the owning surface, the DB state, the Artifact Tree, or the source trail.

Prepared from `10_Operator_Guides(1).md`. Styled after the Semantic Release Edit Contract Handbook template, with clean tables and rendered operator diagrams.

Contents

Recommended first pass for new operators: read sections 1-7, then jump to the workflow or recovery section matching the current task.

1. First read: operator context

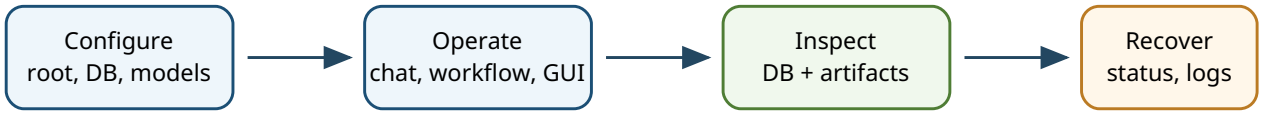
This chapter is the practical operator runbook for The Ontology Machine.

The architecture chapters explain what owns what. This chapter answers the operator questions:

- Which surface should I open?
- Which path should I select?
- What should I see when it works?
- Where do I look when it does not?
- What must I never delete while trying to clean up?

Use this chapter when running the system, preparing a demo, building a corpus, recovering a blocked workflow or handing a machine state to another operator.

> **One sentence to remember**
> Do not guess across ownership boundaries: use Kernel state, Corpus DB state, Artifact Tree evidence, and the agent-visible source trail.



Follow the owner boundary: Kernel state, Corpus DB state, Artifact Tree evidence, or agent source trail.

Figure 1. Minimum operator loop: configure, operate, inspect, then recover through the owning surface.

2. Operator Mental Model

The system has three normal operator surfaces.

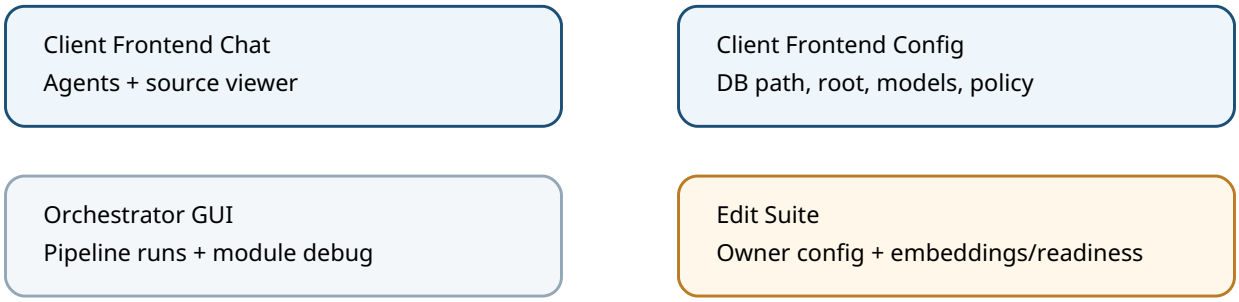


Figure 2. Operator surfaces: chat and config are separate servers; Orchestrator and Edit Suite are owner/support surfaces.

Surface	Start From Source Workspace	Purpose
Client Frontend Chat	`Client Frontend\start.bat`	Query Agent, Ontology Agent, Taxonomy Agent, source viewer
Client Frontend Config	`Client Frontend\config.bat`	DB path, Ontology Machine root, model credentials, prompts, policy
Orchestrator GUI	`00 - Orchestrator\run.bat`	Direct document pipeline runs and module debugging

There is also an operator/support surface:

Surface	Start From Source Workspace	Purpose
Edit Suite	`06 - Edit Suite\run.bat`	Owner config surfaces, Corpus Builder actions, drift/readiness inspection

The Client Frontend chat server and config server are separate processes.

Server	URL	Source Launcher
Chat UI	`http://127.0.0.1:3000`	`Client Frontend\start.bat`
Config UI	`http://127.0.0.1:3001/config`	`Client Frontend\config.bat`

Do not confuse the config server on port `3001` with the chat server on port `3000`. If only the config server is running, the Query/Ontology/Taxonomy agents are not available.

Frontend runtime logs live under:

```
%LOCALAPPDATA%\Enterprise Stack\Client Frontend\logs
```

Important log files:

```
startup.log
startup-browser-helper.log
config-startup.log
config-browser-helper.log
```

3. Path Map

In the source workspace used by this documentation, the root is:

```
C:\Users\Norma\Workspace\The Ontology Machine
```

Use relative paths in this chapter as paths below that root unless a full path is shown.

The bundled demo corpus is:

```
SampleDB\Consciousness Travel - Default Demo
```

The bundled demo DB is:

```
SampleDB\Consciousness Travel - Default Demo\Corpus\corpus.db
```

The normal Artifact Tree layout is:

```
Artifact Tree\
  Input\
  Corpus\
    corpus.db
  Semantic Release\
  Documents\
  Error Cases\
```

Kernel-governed workflows expect the DB to stay inside the selected Artifact Tree's `Corpus` folder. A random SQLite file somewhere else can be readable, but it is not a valid Kernel target for ingest, reset, rebuild or merge.

4. Which Surface Should I Use?

Use the Query Agent when:

- you want answers over an existing DB
- you want coverage counts
- you want to inspect source documents
- you want to compare ontology lenses
- you want source-backed analysis without changing the DB

Use the Ontology Agent when:

- you want to build or refresh the Base Graph
- you want to create a lens
- you want to add evidence-bound interpretation
- you want a correction, audit or review layer
- you want to compare or activate lenses

Use the Taxonomy Agent when:

- you want Kernel-guided database creation
- you want Kernel-guided ingestion
- you want to create a custom taxonomy or projection
- you want to rebuild or merge DBs
- you want to recover a blocked Kernel workflow

Use the Orchestrator directly when:

- you want the classic pipeline GUI
- you want module-stage debugging
- you want direct control over pipeline runs without the Kernel chat surface

Use the Edit Suite when:

- you need owner configuration surfaces
- you need Corpus Builder actions such as `Generate Embeddings`
- you need drift/readiness inspection
- you need to inspect owner-provided edit contracts

5. First Start From Source Workspace

Purpose

Start the system from the source workspace without relying on installed shortcuts or generated installer wrappers.

Steps

1. Open a terminal in:

```
C:\Users\Norma\Workspace\The Ontology Machine
```

2. Start the config UI:

```
Client Frontend\config.bat
```

3. If the browser does not open automatically, open:

```
http://127.0.0.1:3001/config
```

4. Save the required setup values.
5. Start the chat UI in a separate terminal:

```
Client Frontend\start.bat
```

6. If the browser does not open automatically, open:

```
http://127.0.0.1:3000
```

Expected Signs

- Config UI opens on port `3001`.
- Chat UI opens on port `3000`.
- The LLM readiness badge is green when credentials are valid.
- The Base Graph badge is green when the active DB has Base Graph data.
- The Ontology Lens badge shows the current lens count.

Failure Symptoms

Symptom	Meaning	Recovery
Browser does not open	Server may still be running; browser helper failed	Open the URL manually and inspect `*-browser-helper.log`.

Symptom	Meaning	Recovery
Config opens but chat fails	Only port `3001` is running	Start `Client Frontend\start.bat`.
Port cleanup fails	Another process owns the port	Close old Frontend windows/ terminals or inspect the process named in the startup log.
Runtime check fails	Bundled Node/PowerShell runtime is missing or damaged	Rebuild/reinstall the Frontend runtime.

6. Configure Credentials

Purpose

Give the agents and pipeline enough provider access to answer, author, normalize and embed.

Steps

1. Start the config UI:

   ```
   Client Frontend\config.bat
   ```

2. Open the `Setup` tab.
3. Set the Ontology Machine root to the workspace/install root.
4. Set the active Corpus DB path.
5. Open the `Models` tab.
6. Configure the LLM provider and model.
7. Test the LLM connection.
8. Configure the embedding provider and model if semantic/vector search or embedding generation should work.
9. Test the embedding connection.
10. Save.
11. Restart the chat UI if it was already open.

Expected Signs

- LLM test succeeds.
- Embedding test succeeds if an embedding provider is configured.
- Chat UI shows `LLM ready`.
- Query Agent can answer a small question.

Important Distinction

LLM credentials unlock agent answers and LLM-backed pipeline stages.

Embedding credentials unlock vector search and embedding generation.

Missing embeddings do not mean the DB is corrupt. They mean semantic/vector retrieval is degraded and the system will fall back to SQL, document tools or lexical search where possible.

Failure Symptoms

Symptom	Meaning	Recovery
LLM badge red	Chat model is unavailable	Check provider, base URL, model and API key/OAuth state.
Embedding badge red	Embedding provider unavailable	Configure embedding provider or accept vectorless retrieval.
Agent says `Failed to fetch`	Frontend chat server is not reachable	Confirm port `3000` is running and inspect `startup.log`.
Taxonomy Agent cannot call Kernel tools	MCP/Kernel root not resolved	Check Ontology Machine root in Setup.

7. Select A Corpus DB

Purpose

Point the Query and Ontology agents at the intended DB.

Steps

1. Start the config UI.
2. Open `Setup`.
3. Set the Corpus DB path to a real SQLite Corpus DB.
4. Prefer the canonical Artifact Tree location:

   ```
   <Artifact Tree>\Corpus\corpus.db
   ```

5. Save.
6. Restart the chat UI.
7. Ask the Query Agent:

   ```
   Give me a database coverage snapshot.
   ```

Demo DB

For the bundled demo, use:

```
C:\Users\Norma\Workspace\The Ontology Machine\SampleDB\Consciousness Travel - Default Demo\Corpus\corpus.db
```

Expected Signs

- Query Agent can open the DB.
- Coverage snapshot returns document counts.
- Base Graph badge reports ready or missing.
- Ontology Lens badge reports a number.

Failure Symptoms

Symptom	Meaning	Recovery
Agent says no DB configured	Config did not save or chat server was not restarted	Save config and restart chat UI.
DB opens but Kernel workflow rejects it	DB is outside Artifact Tree `Corpus`	Use the DB under the selected Artifact Tree.

Symptom	Meaning	Recovery
Coverage is empty	DB may be empty or reset	Check Artifact Tree and Semantic Release state before ingesting.

8. First Demo Run

Purpose

Prove that the Frontend, DB reader, source viewer and model credentials work before running an expensive ingest.

Preconditions

- Config points at the bundled demo DB.
- LLM credentials are configured.
- Chat UI is running.

Steps

1. Open:

   ```
   http://127.0.0.1:3000
   ```

2. Select the Query Agent.

3. Ask:

   ```
   What is in this corpus? Give me a compact overview and mention the source evidence.
   ```

4. Click a source card.

5. Confirm that the page viewer opens page evidence.

6. Ask:

   ```
   Does this DB have a Base Graph and ontology lenses?
   ```

Expected Signs

- Query Agent answers over the demo corpus.
- Source cards appear.
- Page images open in the viewer.
- Base Graph and lens status are visible without asking the agent.

Failure Symptoms

Symptom	Meaning	Recovery
No answer	LLM not configured or chat server unavailable	Check config and port `3000`.

Symptom	Meaning	Recovery
Answer but no sources	The turn may not have touched resolvable documents	Ask for source-backed evidence or inspect DB coverage.
Source card cannot open image	DB/folder image evidence mismatch	Rebuild or inspect `document_page_images` and `Documents\page_images`.

9. Start The Orchestrator

Purpose

Run the direct pipeline GUI and module debug surfaces.

Steps

1. Open:

```
C:\Users\Norma\Workspace\The Ontology Machine
```

2. Start:

```
00 - Orchestrator\run.bat
```

3. Wait for the GUI to open.
4. Verify that the selected workspace/artifact target is the one you intend to mutate.
5. Verify credentials and runtime settings if the run uses LLM-backed stages.

Expected Signs

- Orchestrator GUI opens.
- Runtime startup checks pass.
- Selected Artifact Tree and DB are visible in the UI.

Failure Symptoms

Symptom	Meaning	Recovery
Bundled Python missing	Runtime was not built or install is damaged	Run module runtime build/reinstall.
Healthcheck fails	A required module/runtime cannot start	Inspect Orchestrator startup output and module health.
Wrong Artifact Tree selected	You may write to the wrong target	Stop before running and select/create the correct tree.

10. Create Or Select An Artifact Tree

Purpose

Prepare the filesystem evidence surface that will hold input, artifacts, Semantic Release and Corpus DB.

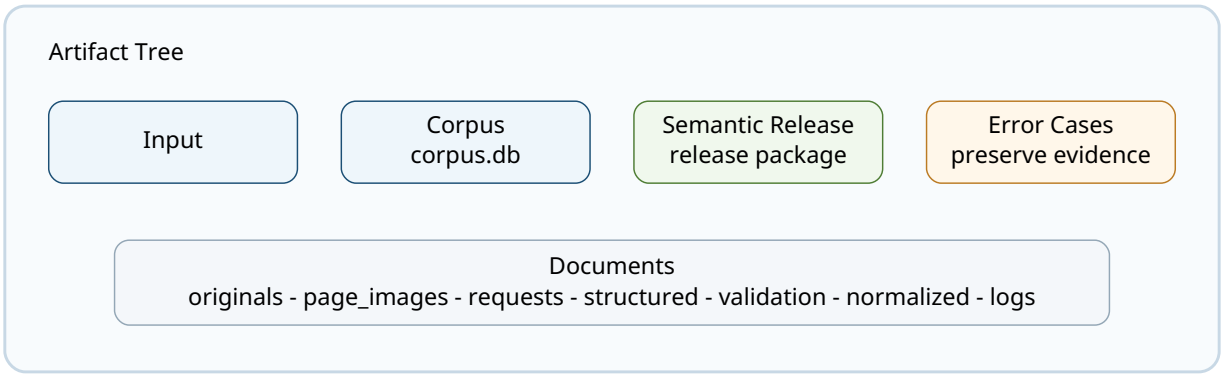


Figure 3. Canonical evidence layout: the Kernel expects the database under the selected Artifact Tree's Corpus folder.

Direct Orchestrator Route

Use the Orchestrator GUI to create or select the target workspace. The Orchestrator can create the canonical folders:

```
Input
Corpus
Semantic Release
Documents\logs
Documents\normalized
Documents\originals
Documents\page_images
Documents\raw_extracts
Documents\requests
Documents\structured
Documents\validation
Error Cases
```

Kernel Route

Use the Taxonomy Agent workflow dropdown to create a DB route. The Kernel will ask for paths, names and confirmations through dialogs.

Expected Signs

- Required folder contract exists.
- `Corpus` contains or will contain `corpus.db`.

- `Semantic Release` contains or will contain a release package.
- `Input` is ready for source files when ingestion is planned.

Do Not

- Do not hand-create half of the folder contract and expect Kernel workflows to trust it.
- Do not move the DB outside `Corpus`.
- Do not flatten merge artifact folders after the fact.

11. Create An Empty DB

Purpose

Create a new Kernel-controlled Corpus DB and optionally attach a default or custom Semantic Release.

Steps

1. Start the Client Frontend chat UI.
2. Select the Taxonomy Agent.
3. Open the workflow dropdown.
4. Choose the creation route that matches the goal:

Goal	Workflow
Empty DB only	`empty_database_no_semantic_release`
Default taxonomy, no projections yet	`empty_database_default_taxonomy_no_projections`
Default taxonomy and projections	`empty_database_default_taxonomy_default_projections`
Default taxonomy, custom projections	`empty_database_default_taxonomy_custom_projections`
Custom taxonomy, no projections yet	`empty_database_custom_taxonomy_no_projections`
Custom taxonomy and custom projections	`empty_database_custom_taxonomy_custom_projections`

5. Answer Kernel dialogs.
6. Wait for the final notice.

Expected Signs

- Artifact Tree exists.
- DB exists under `Corpus`.
- Creation final notice appears in the Taxonomy Agent chat.
- If a complete release was created, DB state is ready for ingest.

Failure Symptoms

Symptom	Meaning	Recovery
Workflow stops incomplete	The selected route intentionally staged taxonomy/projection work	Use the offered continuation workflow.
Missing release on ingest	DB exists but no active Semantic Release exists	Continue/rebuild/activate release before ingest.
Target conflict	A DB or folder already exists	Confirm overwrite only when you really intend it.

12. Create A Custom Release

Purpose

Let the Kernel and LLM-assisted authoring create a taxonomy/projection release from samples instead of using the default taxonomy.

Steps

1. Start the Client Frontend chat UI.
2. Select the Taxonomy Agent.
3. Choose a custom route:

   ```
   empty_database_custom_taxonomy_custom_projections
   ```

   or, for staged work:

   ```
   empty_database_custom_taxonomy_no_projections
   ```

4. Answer sample/input dialogs.
5. Let the Kernel analyze samples.
6. Review generated taxonomy/projection summaries in the final notice.
7. If the workflow staged an incomplete release, use:

   ```
   create_custom_taxonomy_path
   create_custom_projection_path
   ```

   only through the Kernel's resumable/continuation state.

Expected Signs

- `Semantic Release\releases\...\release.json` exists.
- Release identity/fingerprint is recorded.
- DB has the release attached and active when the route completes.
- Final notice says whether the DB is ready for ingest.

Failure Symptoms

Symptom	Meaning	Recovery
Sample analysis fails	Input sample cannot be rendered/ analyzed or request too large	Inspect Kernel blocker and source artifacts.

Symptom	Meaning	Recovery
Projection-free state	Taxonomy exists but projections are incomplete	Continue with `create_custom_projection_path` through resume state.
LLM unavailable	Authoring cannot proceed	Configure LLM credentials and retry/resume.

13. Run Ingestion Through The Taxonomy Agent

Purpose

Run a Kernel-governed ingestion into an existing Artifact Tree and active DB.

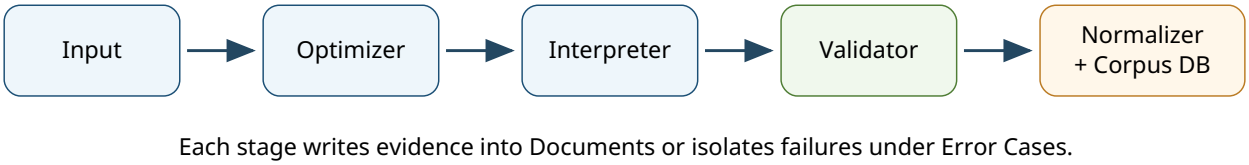


Figure 4. Ingestion spine: stage artifacts are as important as final database rows for debugging and handover.

Preconditions

- Artifact Tree exists.
- Target DB exists under `Corpus`.
- DB has an active Semantic Release.
- Source files are available for the selected input route.
- LLM credentials are configured for Interpreter/Normalizer stages.

Steps

1. Start the Client Frontend chat UI.
2. Select the Taxonomy Agent.
3. Choose:

```
manual_pipeline_run
```

4. Answer Kernel dialogs for target DB, input presence and confirmations.
5. Watch the progress panel.
6. Wait for the final notice.

Expected Signs

- Progress moves through pipeline stages.
- `Documents\requests` receives persisted request payloads.
- `Documents\raw_extracts`, `structured`, `validation` and `normalized` receive stage artifacts.

- `Corpus\corpus.db` receives document rows.
- Final notice reports successes, error cases and target paths.

Failure Symptoms

Symptom	Meaning	Recovery
Missing active Semantic Release	DB cannot materialize normalized documents safely	Activate/create a complete release first.
Progress appears stuck	Long run or event/final notice issue	Ask Taxonomy Agent for `kernel_status`; inspect Frontend logs.
Final notice missing	Kernel mirror event may not have reached chat	Check progress panel, `kernel_status`, and logs.
Owner error	Orchestrator or Corpus Builder failed inside its contract	Inspect owner logs and Error Cases.

14. Run Ingestion Through The Orchestrator

Purpose

Run the classic direct pipeline surface without the Taxonomy Agent.

Preconditions

- Orchestrator GUI is running.
- Artifact Tree is selected.
- DB and active Semantic Release are selected or created.
- Input files are selected or placed in the expected input location.

Steps

1. Start:

```
00 - Orchestrator\run.bat
```

2. Confirm the target Artifact Tree.
3. Confirm target DB and release state.
4. Add/select input files.
5. Start the pipeline.
6. Watch stage progress and logs.
7. Inspect `Documents` and `Error Cases` after completion.

Expected Signs

- Optimizer, Interpreter, Validator, Normalizer and Corpus Builder stages run.
- Successful artifacts move into `Documents`.
- Failed artifacts move into `Error Cases`.
- DB rows appear in `Corpus\corpus.db`.

Failure Symptoms

Symptom	Meaning	Recovery
Request enrichment/interpreter status stuck	UI progress did not receive a later state or finalization stalled	Inspect Orchestrator logs and final stage artifacts.
Unsupported file	Intake route could not classify/process input	Check `Error Cases\Intake` or route diagnostics.

Symptom	Meaning	Recovery
Normalizer/Validator review	The run succeeded but needs human review	Inspect review flags in DB and artifacts.

15. Inspect A Successful Run

Purpose

Prove that a run produced usable corpus state before handing it to another person or starting ontology work.

Checklist

1. `Corpus\corpus.db` exists.
2. `Documents\originals` contains published originals.
3. `Documents\page_images` contains visual ground truth for rendered pages.
4. `Documents\requests` contains OCR/Interpreter/Normalizer requests where those stages were used.
5. `Documents\structured` contains Interpreter output.
6. `Documents\validation` contains Validator reports.
7. `Documents\normalized` contains Normalizer output.
8. `Semantic Release\releases\...\release.json` exists.
9. Query Agent coverage snapshot returns document counts.
10. `Error Cases` is either empty or intentionally preserved.

Useful Query Agent Questions

```
Give me a coverage snapshot for this DB.
How many source documents and materialized page rows are present?
Does this corpus have document_page_images?
Show documents with needs_review flags.
Does this DB have an active Semantic Release?
Does this DB have embeddings?
```

Useful SQL Checks

Run through Query Agent `sql_query` or another read-only SQLite surface:

```
SELECT COUNT(*) AS documents FROM documents;
SELECT COUNT(*) AS page_images FROM document_page_images;
SELECT COUNT(*) AS source_documents FROM source_documents;
SELECT COUNT(*) AS source_pages FROM source_document_pages;
SELECT COUNT(*) AS lenses FROM ontology_lenses;
SELECT COUNT(*) AS needs_review FROM documents WHERE needs_review = 1;
SELECT COUNT(*) AS chunks FROM embedding_chunks;
SELECT COUNT(*) AS legacy_embeddings FROM embeddings;
```

If a table is missing, the DB is old, incomplete or not a current Corpus DB.

16. Inspect Error Cases

Purpose

Understand failed or review-marked evidence without losing the context needed for rerun or support.

Where To Look

Error bundles live under:

```
<Artifact Tree>\Error Cases\
```

Typical stage roots:

```
Error Cases\Intake\
Error Cases\Optimizer\
Error Cases\Interpreter\
Error Cases\Validator\
Error Cases\Normalizer\
Error Cases\Corpus Builder\
```

Typical bundle evidence:

```
originals\
raw_extracts\
page_images\
requests\
structured\
validation\
normalized\
logs\
```

Steps

1. Open `Error Cases`.
2. Identify the stage folder.
3. Open the `.error_manifest.json` under `logs` if present.
4. Compare the request payload, model output and validator report.
5. Check whether the original was moved into the bundle or whether this was a page-scoped failure.
6. Inspect the DB for `needs_review` rows.

Expected Signs

- Error bundle contains enough evidence to explain the failure.

- Review cases that still succeeded remain in `Documents` and are marked in DB.
- Hard failures are isolated in `Error Cases`.

Failure Symptoms

Symptom	Meaning	Recovery
Error bundle has no request	Failure happened before LLM request or request persistence failed	Inspect intake/optimizer logs.
Review flag but no Error Case	This can be valid: review success is not necessarily a hard failure	Inspect DB review reason and successful artifacts.
Original missing from both places	Artifact publication issue	Stop cleanup and inspect logs/ manifests before rerun.

17. Work With Review Flags

Purpose

Separate successful-but-reviewable documents from hard pipeline failures.

Review flags can come from:

- Interpreter review state
- Validator warnings/issues
- Normalizer review state
- malformed or uncertain processing payloads

Steps

1. Ask the Query Agent:

   ```
   List documents with needs_review and explain the review reasons.
   ```

2. Or inspect with SQL:

   ```
   SELECT id, file_name, needs_review, review_reason,
          interpreter_needs_review, interpreter_review_reason,
          normalizer_needs_review, normalizer_review_reason,
          validator_status, validator_issues_count
   FROM documents
   WHERE needs_review = 1
   ORDER BY file_name;
   ```

3. Open the source page image and validation/normalized artifacts.

4. Decide whether the issue is acceptable, a taxonomy/projection weakness, a validator rule issue or an unsupported input pattern.

Expected Signs

- Review rows remain queryable.
- Source evidence remains available.
- The operator can decide whether to accept, ingest again or build a correction ontology lens.

18. Build Or Refresh The Base Graph

Purpose

Turn page-level materialization into source-document grouping and deterministic structural relations.

Steps

1. Start the Client Frontend chat UI.
2. Select the Ontology Agent.
3. Ask:

   > `Run basic_relation_mining for the active DB.`

4. Wait for the report.
5. Confirm the Base Graph badge turns green.
6. Ask the Query Agent for source-document coverage.

Expected Signs

- `source_documents` is populated.
- `source_document_pages` is populated.
- Base Graph badge reports ready.
- Query Agent stops treating page rows as unrelated documents.

Failure Symptoms

Symptom	Meaning	Recovery
Tool says DB path cannot be proven	Active DB/config does not match owner target proof	Check Frontend DB path and Artifact Tree binding.
Base Graph still missing	Mining failed or DB is incompatible	Inspect Ontology Agent result and DB schema.
Source groups look wrong	Materialized `source_document_id`/page fields are wrong	Rebuild or ingest again; do not invent grouping by ontology lens.

19. Create An Ontology Lens

Purpose

Persist evidence-bound knowledge above the materialized corpus without changing base facts.

Steps

1. Confirm the DB is the intended target.
2. Prefer running Base Graph first.
3. Select the Ontology Agent.
4. Describe the lens intent:

   ```
   Build a lens that reads these documents from the perspective of ...
   ```

5. Let the agent inspect relevant documents first.
6. Let it write bounded ontology batches.
7. Wait for validation and embedding status.
8. Ask the Query Agent to use or compare the lens.

The Ontology Agent should normally inspect through compact document views first: summary, ontology evidence, rows or document-level provenance. Full document reads are still available, but should be a last escalation when the compact views do not contain enough evidence.

Expected Signs

- `ontology_lenses` contains the new lens.
- Lens status is `draft`, `ready` or `archived`.
- Activation is stored separately from status.
- Nodes, edges, assertions and evidence links exist.
- Ontology Agent reports validation status.

Failure Symptoms

Symptom	Meaning	Recovery
Missing ID/FK/NOT NULL error	Agent write batch violated schema discipline	Let repair loop run; if exhausted, inspect preflight error.
Embedding warning	Lens exists but ontology vector chunks were not refreshed	Configure embeddings or accept degraded ontology retrieval.
Lens contradicts base facts	This may be correct for correction/ audit lenses	Keep base fact and lens claim separate.

20. Compare Ontology Lenses

Purpose

Understand how different evidence-bound interpretations read the same corpus.

Steps

1. Select the Query Agent.
2. Ask:

```
List active ontology lenses and compare what they emphasize.
```

3. If needed, ask:

```
For each lens, show claims that are evidence-linked and claims that are weakly supported.
```

4. Open source cards for disputed claims.

Expected Signs

- Query Agent distinguishes base facts from lens claims.
- Correction/audit/review lenses are surfaced when they contradict materialized facts.
- Source cards support claims where evidence exists.

Failure Symptoms

Symptom	Meaning	Recovery
Query Agent ignores lenses	Prompt/config drift or lens inactive	Check lens activation and Query prompt policy.
Lens has no evidence links	Ontology is interpretive but weakly grounded	Ask Ontology Agent to add evidence links.
Source link suspicious	Source named in answer was not resolved in turn	Re-ask with explicit evidence and inspect source panel.

21. Regenerate Embeddings

Purpose

Create or refresh vector search data after ingest, rebuild or configuration changes.

Important Boundary

Embedding generation is owner work. The Query Agent cannot create embeddings. The Ontology Agent refreshes ontology embeddings after validated ontology writes, but corpus document embeddings are Corpus Builder work.

Normal Path

Use the Edit Suite:

1. Start:

   ```
   06 - Edit Suite\run.bat
   ```

2. Open the Corpus Builder surfaces.
3. Open `Embeddings Policy`.
4. Verify dimensions, batch size and max text length.
5. Run:

   ```
   Generate Embeddings
   ```

6. Wait for completion.
7. Reopen the Client Frontend and test semantic search.

Rebuild Path

Kernel rebuild can also create embeddings for the rebuilt DB when embedding provider configuration is available.

Use:

```
database_rebuild_from_artifacts
```

through the Taxonomy Agent.

Expected Signs

- `embedding_chunks` and/or `embeddings` contains rows.
- Query Agent semantic search stops warning about missing embeddings.

- Rebuild final notice reports embedding result.

Failure Symptoms

Symptom	Meaning	Recovery
provider skipped	Embedding provider is not configured	Configure embedding provider and rerun.
Count remains zero	No pending sources or action targeted wrong DB	Confirm active DB and whether existing vectors need full rebuild.
Dimension error	Model dimensions do not match DB/ config policy	Align embedding model and policy before regenerating.

Operator Note

`Generate Embeddings` computes embeddings for pending sources. It is not a universal "delete all vectors and rebuild everything" button unless the owner workflow explicitly clears/rebuilds the DB. For a clean full refresh, rebuild from artifacts or use an owner-approved clear/regenerate path.

22. Rebuild A DB From Artifacts

Purpose

Recreate a Corpus DB from the Artifact Tree evidence and active Semantic Release package.

Use When

- DB was deleted or corrupted.
- DB schema changed and rebuild is safer than migration.
- You need to prove the artifact evidence can recreate materialized state.
- You want a clean DB after development/testing churn.

Preconditions

- Artifact Tree folder contract exists.
- `Documents\normalized` contains normalized artifacts that can be rebuilt.
- `Semantic Release\releases\...\release.json` exists and is complete.
- Page images/originals are still present.

Steps

1. Start the Client Frontend chat UI.
2. Select the Taxonomy Agent.
3. Choose:

   ```
   database_rebuild_from_artifacts
   ```

4. Select the Artifact Tree when the Kernel asks.
5. Choose/confirm target DB name.
6. Confirm overwrite if required.
7. Wait for final notice.
8. Run Base Graph mining again if the rebuilt DB needs source-document relations.
9. Regenerate embeddings if required.

Expected Signs

- New/rebuilt DB exists under `Corpus`.
- Semantic Release is loaded and activated.
- Rebuild manifest exists under `Documents\logs\rebuild_runs`.

- Final notice reports record count and embedding result.

Failure Symptoms

Symptom	Meaning	Recovery
`release_missing`	No complete Semantic Release package in Artifact Tree	Restore/recreate release before rebuild.
`release_incomplete`	Release identity/fingerprint incomplete	Inspect `release.json` and release artifacts.
`target_conflict`	Target DB exists	Confirm overwrite only when intended.
Embedding unavailable	Rebuild succeeded but vectors were skipped	Configure embedding provider and generate embeddings.

23. Merge Databases

Purpose

Create one additive target DB from two or more source Artifact Trees/DBs while preserving source identity, release identity, artifacts, Base Graph and ontology layers.

Use When

- multiple corpora should become one queryable DB
- sample DBs should be combined for a demo
- separately ingested batches need one shared search surface

Preconditions

- Source Artifact Trees are intact.
- Source DBs are under their own `Corpus` folders.
- Source Semantic Releases are complete.
- Target Artifact Tree can be created or selected.
- Enough disk space exists for copied artifacts and merged DB.

Steps

1. Start the Client Frontend chat UI.
2. Select the Taxonomy Agent.
3. Choose:

   ```
   database_merge_additive_only
   ```

4. Select source DBs/Artifact Trees through Kernel dialogs.
5. Select or create target Artifact Tree.
6. Confirm choices.
7. Wait for final notice.
8. Inspect target `Documents\logs\merge_runs`.
9. Ask Query Agent for merged coverage.

Expected Signs

- Target Artifact Tree exists.
- Target DB exists under target `Corpus`.
- Target Semantic Release is attached and active.

- Source artifacts are copied collision-safely.
- Merge logs/manifests exist.
- Source document identities remain distinguishable.
- Ontology lenses from source DBs are preserved/remapped when valid.

Failure Symptoms

Symptom	Meaning	Recovery
`too many SQL variables`	Merge batching bug or too-large statement	Use fixed build with batched SQL remap.
response too large	Adapter returned excessive payload	Use fixed build with compact owner responses.
parent cycle in ontology lenses	Source ontology parent references are invalid/cyclic	Inspect source lenses before merge.
Source artifacts look nested/ugly	Collision-safe copy names may be intentional	Do not flatten manually; check merge manifest.

24. Reset A Database

Purpose

Clear materialized corpus rows while preserving the active Semantic Release.

Use When

- you want to ingest again into the same release context
- a batch should be cleared without deleting the release package
- Kernel recovery recommends reset

Steps

1. Select the Taxonomy Agent.
2. Choose:

   ```
   reset_database
   ```

3. Answer destructive confirmation dialogs.
4. Wait for final notice.
5. Verify that the release state is preserved.

Expected Signs

- Materialized document rows are cleared.
- Active Semantic Release remains attached/active.
- DB is ready for a planned next ingest if release state is complete.

Do Not

- Do not delete the DB file manually as a substitute for reset.
- Do not reset the wrong target because two DBs have similar names.
- Do not reset while a pipeline run is active.

25. Debug A Stuck Kernel Workflow

Purpose

Find out whether a Kernel workflow is running, waiting, blocked, resumable or stale.

Steps

1. Select the Taxonomy Agent.
2. Ask:

```
Check kernel_status.
```

3. If a dialog is pending, answer it in the Kernel dialog panel.
4. If a run is resumable, ask:

```
Show kernel_resume_state.
```

5. Continue only with an offered `resume_option_ref`.
6. If the run should stop, ask for:

```
kernel_cancel_active_run
```

7. Inspect Frontend logs if the UI does not match Kernel state.

Expected Signs

- Active run, pending interaction or blocked state is visible.
- Recovery options are opaque references, not raw guessed IDs.
- Final notice or blocker explanation appears after recovery.

Failure Symptoms

Symptom	Meaning	Recovery
Background process running for too long	Long workflow or stale state	Check `kernel_status`; do not start duplicate workflow blindly.
Pending dialog not visible	Event bridge/UI is out of sync	Refresh UI, inspect logs, ask status again.
Resume option stale	Kernel state changed after option was listed	Refresh resume state and use current option.
Owner error	Kernel reached owner module and owner failed	Inspect owner module logs and artifacts.

26. Inspect Source Links In Query Answers

Purpose

Catch source hallucination or context leakage without manually counting every citation.

Steps

1. Ask the Query Agent for a source-backed answer.
2. Compare answer citations/file mentions with the source column.
3. Click source buttons.
4. Treat red/suspicious source markers as a warning.

Meaning

A suspicious source marker can mean:

- the model named a source it did not read in the current turn
- the model leaked a source from previous chat context
- source reconciliation failed to match a valid mention

It does not automatically prove the answer is false, but it makes the claim review-worthy.

27. Clean Temporary Files Safely

Purpose

Remove runtime trash without damaging evidence or the ability to rebuild.

Before Cleaning

1. Close Client Frontend chat/config windows.
2. Close Orchestrator.
3. Close Edit Suite.
4. Confirm no pipeline, rebuild or merge is active.
5. Do not delete SQLite sidecars while the DB is open.

Usually Safe To Delete

- old Frontend startup logs under `%LOCALAPPDATA%\Enterprise Stack\Client Frontend\logs`
- stale process logs outside Artifact Trees
- empty folders not required by the Artifact Tree contract
- stale `*.db-wal` and `*.db-shm` only when the DB is closed and checkpointed
- duplicate external input copies after originals are safely published

Delete Only With Intent

- `Input` contents after a completed run
- old merge receipts
- old pipeline batch logs
- demo Error Cases that should not ship

Do Not Delete In Handover Trees

- `Documents\originals`
- `Documents\page_images`
- `Documents\requests`
- `Documents\structured`
- `Documents\validation`
- `Documents\normalized`
- `Semantic Release`
- active `Corpus\corpus.db`
- ontology-bearing DBs
- current batch manifests
- merge/rebuild manifests

- Error Cases that explain known failures

Rule

Never clean by age alone. In this system, a months-old release package may be more important than a fresh log file.

28. Prepare A Handover Bundle

Purpose

Give another operator enough evidence to query, inspect, rebuild and continue work.

Minimum Bundle

- Artifact Tree root
- `Corpus\corpus.db`
- `Corpus\*.db-wal` and `*.db-shm` only if DB was not cleanly closed
- `Semantic Release`
- `Documents\originals`
- `Documents\page_images`
- `Documents\requests`
- `Documents\structured`
- `Documents\validation`
- `Documents\normalized`
- `Documents\logs`
- `Error Cases`
- notes about active lenses, known review flags and known unsupported inputs

Pre-Handover Checks

1. Query Agent coverage snapshot works.
2. Source viewer can open at least one page image.
3. Semantic Release package exists.
4. Base Graph status is known.
5. Ontology lens count is known.
6. Error Cases are either explained or intentionally absent.
7. Embedding state is known.
8. Last run final notice or Orchestrator completion state is known.

29. Operator Triage Table

Use this table when something looks wrong and you need a first move.

Symptom	First Move	Likely Owner
Chat says `Failed to fetch`	Check whether port `3000` chat server is running	Client Frontend
Config opens but agent does not answer	Start chat server, not config server	Client Frontend
LLM ready is red	Test model credentials in Config	Client Frontend credentials
Embeddings missing	Configure embedding provider or run Generate Embeddings	Corpus Builder / Edit Suite
Ingest says no active Semantic Release	Inspect DB release state and creation route	Corpus Builder / Kernel
DB path rejected by Kernel	Confirm DB is under selected `Corpus` folder	Kernel target identity
Progress panel appears stuck	Ask `kernel_status` and inspect final notices	Semantic Control Kernel
Error Cases present	Inspect stage bundle and manifest	Orchestrator stage owner
Query treats pages as separate docs	Run `basic_relation_mining`	Ontology Agent / Corpus Builder
Ontology write fails	Inspect preflight/validation result	Ontology Agent
Merge fails	Inspect merge final notice and target logs	Kernel / Corpus Builder merge
Source link looks suspicious	Re-ask with explicit evidence and inspect source panel	Query Agent / Frontend source reconciliation

30. Golden Operator Rule

> Do not guess across ownership boundaries.
>
> If the Kernel blocks, use Kernel status and recovery tools.
>
> If Corpus DB materialization is wrong, inspect Corpus Builder artifacts and DB state.
>
> If source evidence is missing, inspect the Artifact Tree.
>
> If an agent answer looks wrong, inspect the tool-visible source trail.
>
> If a run is still active, do not clean, reset, merge or rebuild the same target until the active state is completed, cancelled or explicitly blocked.